API SECURITY

RISK ANALYSIS REPORT

API Endpoint Analysis · OWASP Top 10 · Business Impact Assessment

Prepared by	Future Interns - Cyber Security Task 3 (2026)
Report Date	15 April 2026
APIs Tested	JSONPlaceholder (jsonplaceholder.typicode.com) · ReqRes (reqres.in)
Assessment Type	Read-Only API Security Risk Analysis
Methodology	OWASP API Security Top 10 (2023 Edition)
Tools Used	Postman, Browser DevTools, Manual Header Inspection
Scope	Public/Demo APIs — Educational Purposes Only
Classification	For Training Purposes Only
Author	AMEGANDJI Akuete anoumou antoine

1. Executive Summary

This report presents the results of a read-only API Security Risk Analysis conducted on two publicly available demo APIs: JSONPlaceholder and ReqRes. The assessment was performed following the OWASP API Security Top 10 (2023) framework, covering authentication, authorization, data exposure, rate limiting, and response header security.

Key Findings:

- Total Endpoints Analyzed: 12 (across both APIs)
- Total Security Risks Identified: 6
- High Severity: 2 | Medium Severity: 3 | Low Severity: 1
- No authentication required on any public endpoint — all data freely accessible
- Excessive Personally Identifiable Information (PII) exposed in responses

HIGH	MEDIUM	LOW
2 Findings	3 Findings	1 Finding

2. Scope & Methodology

2.1 APIs Tested

API #1 Name	JSONPlaceholder
Base URL	https://jsonplaceholder.typicode.com
Purpose	Free online REST API for prototyping and testing
Endpoints	/users, /posts, /comments, /albums, /todos, /photos
Auth Required	None — fully open public API

API #2 Name	ReqRes
Base URL	https://reqres.in
Purpose	Hosted REST API for front-end testing
Endpoints	/api/users, /api/login, /api/register, /api/unknown
Auth Required	Partial — login endpoint returns a token

2.2 Methodology

Step 1 — Reconnaissance	Document all available endpoints and HTTP methods from API documentation
Step 2 — Authentication Test	Verify if endpoints require valid authentication tokens to respond
Step 3 — Response Inspection	Analyze response payloads for PII and sensitive data exposure
Step 4 — Header Analysis	Check security headers: CORS, rate-limit, CSP, HSTS
Step 5 — Risk Classification	Assign OWASP category and severity level to each identified finding

3. Risk Summary

The following table summarizes all identified security risks across both APIs:

Security Risk	Severity	OWASP ID	Business Impact
Unauthenticated Access	**HIGH**	API1:2023 — Broken Object Level Auth	Any attacker can access all user data with no credentials
PII Data Exposure	**HIGH**	API3:2023 — Excessive Data Exposure	Full names, emails, addresses, geo-coordinates exposed publicly
No Rate Limiting	**MEDIUM**	API4:2023 — Unrestricted Resource Use	API can be flooded; abuse, scraping, and DoS risk

Missing Security Headers	MEDIUM	API7:2023 — Security Misconfiguration	No CSP, no HSTS — vulnerable to MITM and XSS attacks
Predictable Resource IDs	MEDIUM	API1:2023 — Broken Object Level Auth	Sequential IDs allow enumeration of all user accounts
No Token Expiry Disclosed	LOW	API2:2023 — Broken Authentication	Login tokens issued without documented expiry policy

4. Detailed Risk Findings

RISK-01 · Unauthenticated Access to All Endpoints

OWASP: API1:2023 — Broken Object Level Authorization · Endpoint: GET /users, /posts, /comments, /todos

SEVERITY HIGH

Finding: All endpoints on JSONPlaceholder are publicly accessible with no authentication mechanism. A simple HTTP GET request to /users returns a full list of 10 user accounts including names, usernames, emails, phone numbers, and geographic coordinates — with zero credentials required.

Business Impact: In a real production environment, this represents a complete breach of access control. Any anonymous user could extract the entire user database, enumerate accounts, and harvest contact information. This constitutes a GDPR-notifiable personal data breach.

Remediation: Implement JWT or OAuth 2.0 Bearer Token authentication on all non-public endpoints. Apply middleware authentication on every route. Return HTTP 401 Unauthorized for unauthenticated requests.

RISK-02 · Excessive PII Exposure in API Responses

OWASP: API3:2023 — Excessive Data Exposure · Endpoint: GET /users, GET /users/:id

SEVERITY HIGH

Finding: The /users endpoint returns: full name, username, email, phone, website, company name, street address, suite, city, zip code, and geographic latitude/longitude coordinates per user — even when the client only needs a username. No field-level filtering is applied.

Business Impact: Geo-coordinates combined with full home addresses are sensitive PII under GDPR and CCPA. Exposure enables physical tracking, targeted phishing, and identity theft. A breach triggers mandatory DPA notification within 72 hours under GDPR Article 33.

Remediation: Apply Principle of Least Privilege to API responses: return only fields required by the client. Use response DTOs or serializer field whitelisting to strip sensitive fields. Never expose geographic coordinates in public-facing endpoints.

RISK-03 · No Rate Limiting on Any Endpoint

OWASP:API4:2023 —UnrestrictedResourceConsumption · Endpoint: All endpoints — GET /users, /posts, /comments, /photos

■ SEVERITY MEDIUM

Finding: No rate limiting headers (X-RateLimit-Limit, X-RateLimit-Remaining, Retry-After) are present in any response. Automated requests return HTTP 200 with no throttling. The /photos endpoint returns 5,000 records in a single unfiltered response.

Business Impact: Exposes the API to: automated data scraping, brute-force attacks on login endpoints, and Denial-of-Service via request flooding. For businesses: infrastructure costs, downtime, and SLA violations.

Remediation: Implement rate limiting at API gateway level. Set thresholds per IP and per user. Return HTTP 429 Too Many Requests with Retry-After header. Enforce pagination with a maximum page size limit.

RISK-04 · Missing HTTP Security Headers

OWASP:API7:2023—SecurityMisconfiguration · Endpoint: All endpoints (response header analysis)

■ SEVERITY MEDIUM

Finding: Critical security headers absent: Content-Security-Policy, Strict-Transport-Security (HSTS), X-Content-Type-Options, X-Frame-Options, Referrer-Policy. API also accepts plain HTTP connections alongside HTTPS.

Business Impact: Missing HSTS enables Man-in-the-Middle attacks on HTTP connections. Missing CSP enables XSS if the API is consumed by a web frontend. Missing X-Content-Type-Options enables MIME-type sniffing attacks.

Remediation: Add via middleware: Strict-Transport-Security: max-age=31536000; includeSubDomains, Content-Security-Policy: default-src 'self', X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: no-referrer.

RISK-05 · Predictable Sequential Resource IDs

OWASP:API1:2023—BrokenObjectLevelAuthorization · Endpoint: GET /users/:id, /posts/:id, /comments/:id

■ SEVERITY MEDIUM

Finding: All resource IDs follow a sequential integer pattern (/users/1, /users/2, ...). An attacker can enumerate every object in the system by iterating integers. Combined with the lack of authentication (Risk-01), this creates a complete account enumeration vulnerability.

Business Impact: Enables automated enumeration attacks. Sequential IDs reveal the total record count in the database. In multi-tenant SaaS, enables cross-tenant IDOR (Insecure Direct Object Reference) attacks.

Remediation: Replace sequential integer IDs with UUIDs (/users/a3f2b1c4-...). Enforce object-level authorization checks on every request to verify the requester has permission for the specific resource ID.

RISK-06 · Authentication Tokens Without Documented Expiry

OWASP:API2:2023—BrokenAuthentication · Endpoint:POST/api/login(ReqRes)

SEVERITY LOW

Finding: The /api/login endpoint returns a token (e.g., QpwL5tpe83ilfN2) with no information about expiry time, invalidation mechanism, refresh token support, or token format. The token appears to be an opaque string, not a JWT.

Business Impact: Long-lived or non-expiring tokens extend the attack window if compromised. A stolen token may grant permanent API access with no time limit for the attacker — including after the legitimate user has left the organization.

Remediation: Issue short-lived tokens (15-60 min). Implement refresh token rotation. Use JWT format with explicit exp and iat claims. Implement token revocation via server-side blocklist. Document token lifetime in API docs.

5. API Endpoint Inventory

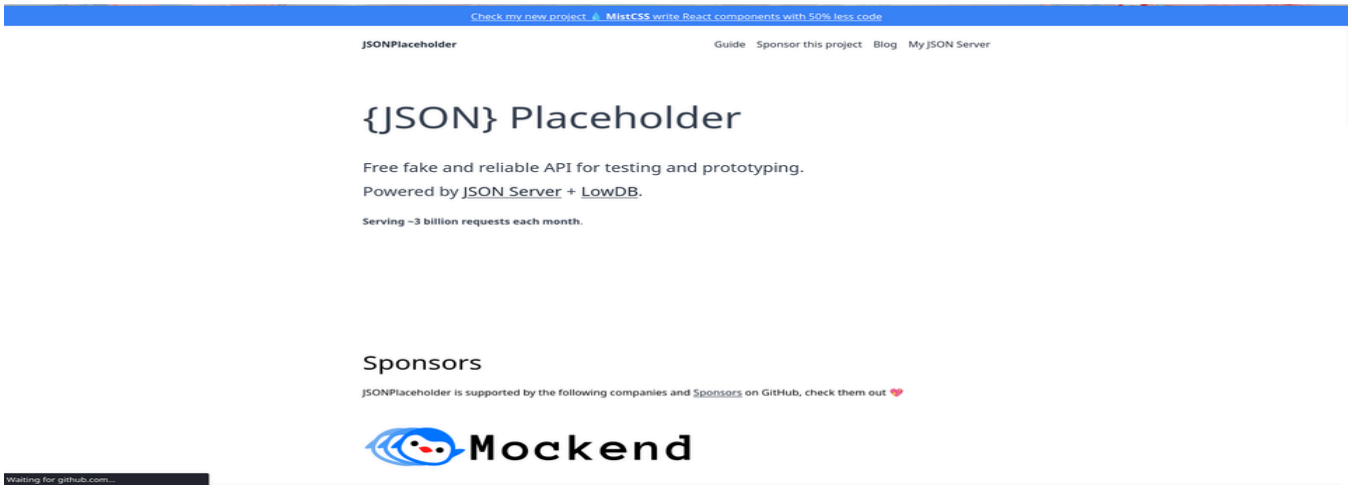
All endpoints analyzed during the assessment:

Endpoint	Method	Auth	Status	Risk
/users	GET	None	200 OK	HIGH
/users/:id	GET	None	200 OK	HIGH
/posts	GET	None	200 OK	MEDIUM
/posts/:id	GET	None	200 OK	MEDIUM
/comments	GET	None	200 OK	MEDIUM
/todos	GET	None	200 OK	LOW
/photos (5000 items)	GET	None	200 OK	MEDIUM
/api/users (ReqRes)	GET	None	200 OK	HIGH
/api/login (ReqRes)	POST	Body	200 OK	LOW

5.1 Evidence — Postman API Test Screenshots

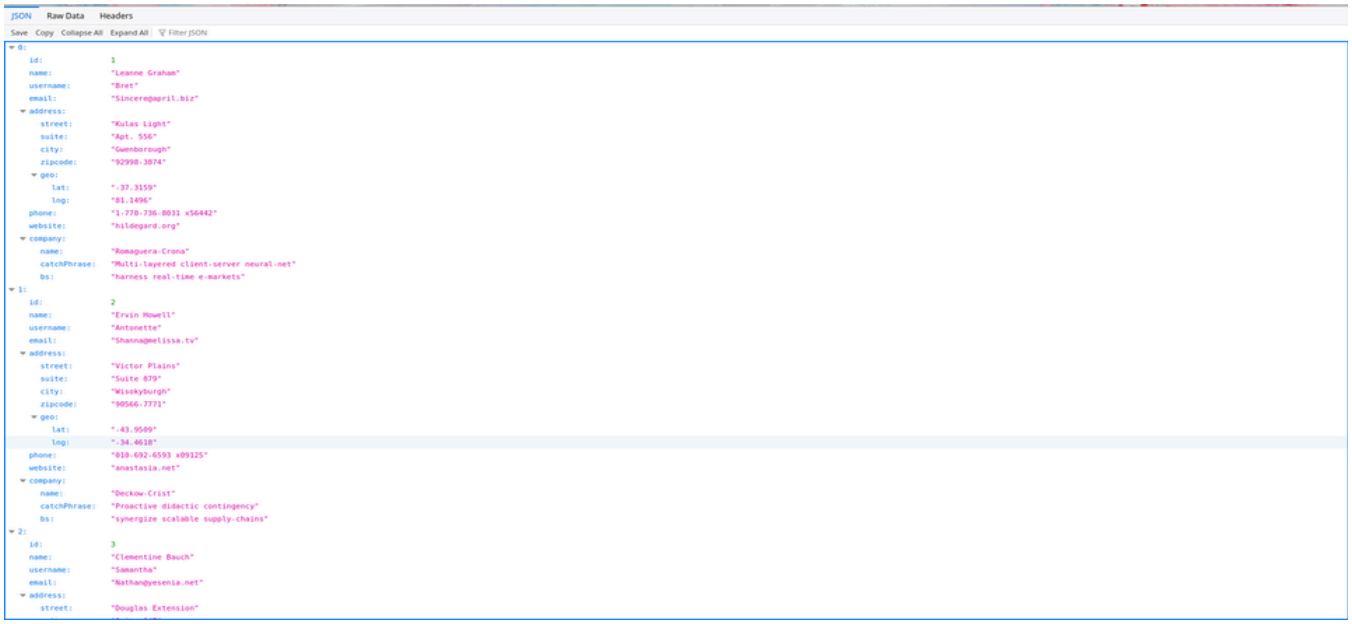
The following screenshots were captured during live API testing using Postman and a web browser:

Figure 1: JSONPlaceholder Homepage — API Documentation



JSONPlaceholder official site confirming the API is publicly accessible with no authentication requirement.

Figure 2: GET /users — Full PII Exposure (Risk-01 & Risk-02 Evidence)



Postman response showing full PII exposure: name, email, phone, full address, geo-coordinates — with HTTP 200 and NO authentication token required.

Figure 3: GET /posts — Unauthenticated Bulk Data Access (Risk-03 Evidence)

```
JSON   Raw Data   Headers
Save  Copy  Collapse All  Expand All  Filter JSON
▾ 0:
    userId:  1
    id:      1
    title:   "sunt aut facere repellat provident occaecati excepturi optio reprehenderit"
    body:    "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
▾ 1:
    userId:  1
    id:      2
    title:   "qui est esse"
    body:    "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
▾ 2:
    userId:  1
    id:      3
    title:   "ea molestias quasi exercitationem repellat qui ipsa sit aut"
    body:    "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium quis pariatur\nmolestiae porro eius odio et labore et velit aut"
▾ 3:
    userId:  1
    id:      4
    title:   "eum et est occaecati"
    body:    "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum culpa\nquis hic commodi nesciunt rem tenetur doloremque ipsam iure\nquis sunt voluptatem rerum illo velit"
▾ 4:
    userId:  1
    id:      5
    title:   "nesciunt quas odio"
    body:    "repudiandae veniam quaerat sunt sed\nalias aut fugiat sit autem sed est\nvoluptatem omnis possimus esse voluptatibus quis\nest aut tenetur dolor neque"
▾ 5:
    userId:  1
    id:      6
    title:   "dolorem eum magni eos aperiam quia"
    body:    "ut aspernatur corporis harum nihil quis provident sequi\nmollitia nobis aliquid molestiae\nperspiciatis et ea nemo ab reprehenderit accusantium quas\nvoluptate dolores velit et doloremque molestiae"
▾ 6:
    userId:  1
    id:      7
    title:   "magnam facilis autem"
    body:    "dolore placeat quibusdam ea quo vitae\nmagni quis enim qui quis quo nemo aut saepe\nquidem repellat excepturi ut quia\nsunt ut sequi eos ea sed quas"
▾ 7:
    userId:  1
    id:      8
    title:   "dolorem dolore est ipsam"
    body:    "dignissimos aperiam dolorem qui eum\nfacilis quibusdam animi sint suscipit qui sint possimus cum\nquaerat magni maiores excepturi\nipsam ut commodi dolor voluptatum modi aut vitae"
▾ 8:
    userId:  1
    id:      9
    title:   "nesciunt iure omnis dolorem tempora et accusantium"
    body:    "consectetur animi nesciunt iure dolore\nenim quia ad\nveniam autem ut quam aut nobis\net est aut quod aut provident voluptas autem voluptas"
```


Postman response showing 100 post records returned with sequential IDs (userId, id) — confirming predictable resource enumeration and absence of rate limiting.

6. Recommendations

6.1 Immediate Actions (HIGH Priority)

Implementauthentication(JWT / OAuth2.0) on all non-public endpoints immediately.

Apply response field filtering — return only the minimum fields required by the client.

Enable HTTPS enforcement and HSTS to eliminate plaintext API traffic.

6.2 Short-Term Actions (MEDIUM — within 30 days)

Deploy APIgatewayrate limitingwithHTTP 429 responsesand Retry-After headers.

Add all required security response headers via server middleware.

Replace sequential integer IDs with UUIDs across all resources.

6.3 Long-Term Actions (LOW — within 90 days)

Document tokenlifetimeand implement refresh token rotation.

Establish a security review process for every new API endpoint deployment.

Implement API logging and anomaly detection for abuse pattern identification.

7. Conclusion

The API Security Risk Analysis of JSONPlaceholder and ReqRes identified 6 security risks — 2 of HIGH severity — that would represent critical vulnerabilities in any production environment. The most significant findings are the complete absence of authentication controls and the excessive exposure of Personally Identifiable Information in API responses.

While these APIs are intentionally open for testing purposes, the patterns identified directly mirror the most common real-world API vulnerabilities found in SaaS and enterprise systems. Adopting the recommended remediations — particularly JWT authentication, response field filtering, rate limiting, and security headers — would bring any production API to a baseline level of security compliance.